TokenTrim AI Prompt Optimizer For Helping Large Language Models Run Faster and Use Energy

Muthu Kumar M
Computer Science and Engineering
Dr.N. G. P. Institute of Technology
Coimbatore, Tamilnadu, India
muthukumar82508@gmail.com

Dr.J.Poongodi
Assistant Professor[SS]
Computer Science and Engineering
Dr.N. G. P. Institute of Technology
Coimbatore, Tamilnadu, India
poongodi.j@drngpit.ac.in

Nagu Ganesh V R
Computer Science and Engineering
Dr.N. G. P. Institute of Technology
Coimbatore, Tamilnadu, India
vrnaguganesh@gmail.com

Navendhiran M
Computer Science and Engineering
Dr.N. G. P. Institute of Technology
Coimbatore, Tamilnadu, India
navendhiranmoorthi1181@gmail.com

Abstract- Large Language Models (LLMs) process every input token during an inference request. In real-world use prompts often include repeated instructions, filler phrases, extra context or winded descriptions that don't really change the task. These elements increase count add to the computational load, raise latency and drive up usage costs. TokenTrim is a framework built at the application layer to reduce input before it reaches the target LLM. It works by taking a user prompt measuring how tokens it uses identifying redundant parts and creating a shorter version. The system then checks if the meaning stays the same through validation. If the optimization isn't reliable it safely reverts to the prompt.

The system's architecture includes components: a browser extension, a Node.js API, a Python-based machine learning service, a token analyzer, a prompt optimizer, a semantic validator, the target LLM and an analytics module. These parts work together to make the optimization process smooth and reliable. The project builds on research in prompt compression, such as LLMLingua, LongLLMLingua and LLMLingua-2 but takes the idea further by turning it into a practical user-friendly workflow. It also follows Green AI principles. Reducing input means less computation during inference. When the cost of optimization is lower than the savings, in compute the system helps cut down on energy use lowers latency reduces costs and decreases carbon emissions. This makes it not only more efficient but more environmentally responsible.

Keywords- Large Language Models, Token Optimization, Prompt Compression, NLP, LLM Inference, Green AI, Carbon Emissions, Semantic Validation, Computational Efficiency..

I. INTRODUCTION

Generative AI systems have quickly become a part of software development, education, research, customer support, content creation and information search. As these systems become more common the amount of inference requests also grows. Each request contains an input context that is made up of tokens. While some context is needed for generation many user prompts contain extra words that are not needed.

The problem is not that prompts can be long. The real challenge is to find which parts of a prompt can be removed without changing the task that the user wants. Manual shortening is not reliable and many prompt improvement systems focus on making prompts easier to read of making them shorter. TokenTrim solves this problem by adding an optimization layer between the user and the LLM.

Research on compression shows that large reductions are possible while keeping useful information. LLMLingua introduced a coarse-to-fine method. Found high compression ratios with only small drops in performance in LLMLinguas tests [1]. Long LLM Lingua extended compression to longer contexts and looked at token, cost and latency reductions [2]. LLMLingua-2 treated compression as classification and stressed efficient and faithful compression [3].

Users often send prompts that're long and contain extra phrases, repeated demands, conversational fillers and context that has little value for the main task. Sending these tokens to the LLM can increase the work that the LLM has to do. The project asks: how can unnecessary prompt tokens be cut while keeping the meaning clear and keeping the output quality good?

Main Contributions of the Proposed System

The key contributions of this research are as follows:

- A framework for token-aware prompt optimization that sits on the application layer.
- A workflow that combines counting, detection of redundant parts, optimization and checking that meaning is preserved.
- A safe fallback that stops an optimization from stopping the user.
- Clear reporting of how original tokens there were, how many after optimization and the percentage of reduction.

- An evaluation that looks at sustainability by linking reduction, to cost, latency, energy use and possible carbon impact.

Section II reviews the related literature, Section III explains the proposed methodology, Section IV presents the experimental analysis and results, and the final section summarizes the study and discusses future work.

II. Related Work

Research today is divided into a clear areas: prompt compression, efficient LLM inference and sustainable AI. I find that LLM Lingua [1] shows that prompt compression can be performed by estimating the importance of each token and, by setting a compression budget. Long LLM Lingua [2] focuses on long-context situations and reports that it can cut both cost and latency. LLMLingua-2 [3] uses data distillation and token classification making compression faster and more faithful.

A. Prompt Compression

Prompt compression methods try to keep the parts of a task while deleting the tokens that add little value. Prompt compression methods are very relevant, to TokenTrim because Prompt compression methods show that prompt length can be shortened without losing much performance.

B. Green AI and Inference Efficiency

Schwartz and his team introduced Green AI as a way of looking at things where using computer power is just as important as how good the model is [4]. Strubell and her group looked at the energy use. Rules around deep learning, in language processing [5]. Luccioni and her team checked how much carbon is used by BLOOM, including the energy and carbon needed when its running [6]. These studies show that we need to look at TokenTrim than just how many tokens it uses.

C. Research Gap

- Research prompts are often shown as models or steps in a process of easy tools for people to use.
- People usually do not know how many words or characters their prompts use.
- Optimizing prompts needs a way to check if the meaning is right because shorter text is not always better.
- A solution, at the application level could work with big language models without changing the model itself.
- The tool that makes prompts better must be tested to see if it is actually helpful because making it work takes some computing power.

Comparison of Existing Approaches

Focus Area	Typical Methodology	Limitation
Reducing unnecessary words in prompts	Users manually rewrite and shorten prompts	Time-consuming and dependent on user expertise
Improving prompt quality and model responses	Uses carefully designed instructions, examples, and formatting	Does not specifically minimize token consumption
Prompt compression and token reduction	Uses language-model-based token-level compression to remove less important tokens	Compression may require additional model processing and specialized integration
Long-context prompt compression	Compresses long prompts while preserving important context	Primarily focused on long-context scenarios
Efficient and faithful prompt compression	Uses token classification to identify and retain important tokens	Requires an additional compression model and processing pipeline
Efficient and faithful prompt compression	Quantization, pruning, distillation, and efficient model architectures	Requires access to or modification of the model infrastructure
User-level token optimization	Analyzes prompts, detects redundancy, compresses unnecessary content, validates semantic similarity, and reports token savings before LLM inference	Optimization quality depends on the tokenizer, semantic model, and optimization strategy

III. PROPOSED METHODOLOGY

A. Prompt Intake and Preprocessing

The system receives the prompt and preserves it as the reference version. Preprocessing may normalize whitespace identify repeated phrases, instructions from supporting context and prepare the prompt for token analysis. Preprocessing must not remove domain- terms or user constraints merely because they are uncommon.

B. Token Analysis

Token savings and reduction are calculated as:

$$Token\ Savings = T_o - T_c$$

$$Reduction\ (\%) = \frac{T_o - T_c}{T_o} \times 100$$

Where:

- T_o = Original Token Count
- T_c = Optimized Token Count
- Token Savings = Original Token Count − Compressed Token Count
- Reduction (%) = Percentage of Tokens Reduced

C. *Redundancy Detection and Optimization*

The optimizer finds repeated instructions, phrases, extra words and information that does not help with the task. Important parts like names, rules, facts, numbers, output needs and how to format are kept. The end result is a prompt that tries to keep the same task definition.

D. *Semantic Validation*

The optimized prompt is checked against the prompt using a way to measure how similar they are. If S(P P') is the similarity score. T is the lowest score that is acceptable the optimized prompt is approved when $S(P,P') \geq \tau$. Checking similarity does not mean the model will act the same. The quality of the response should also be checked.

E. *Safe Fallback*

If the tokenizer, the model that checks meaning the optimizer or the machine learning service has a problem TokenTrim should not stop the user. The original prompt is given back. This is important, for use because optimization should be a choice, not something that can break everything.

F. *Error Handling*

- Tokenizer failure Give back the prompt in a safe way.
- Embedding failure Use another way to check or give back the prompt.
- Model loading failure Check the system. Switch to a backup.
- API timeout Handle the time and try again safely.
- ML service unavailable Skip the optimizer. Use the original prompt.

IV. System Architecture

A. *System Architecture and Design Framework*

The TokenTrim framework has five layers: user interaction, request routing, machine-learning optimization, validation and target inference. The browser extension takes the prompt. Shows optimization details. The Node.js API handles requests and talks, to the Python ML service. The Python service does token analysis, optimization and semantic checks. The prompt that passes validation is sent to the target LLM.

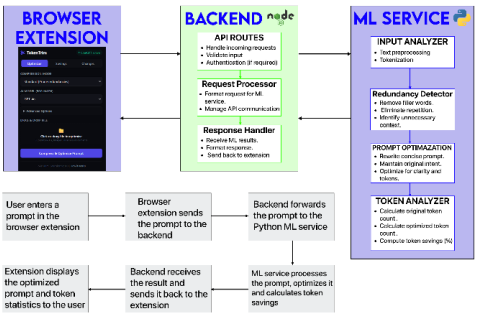


Fig. 1. Overall Architecture of the Proposed Framework

B. *Frontend Layer*

The frontend is the part of the system that lets the use interact. The frontend captures the prompt that the user types. The frontend shows both the version and the optimized version of the text. The frontend also tells the user how many token savings have been made.

C. *Backend and ML Layer*

The frontend is the surface where the user interacts.

The frontend captures the prompt. Displays both the original and the optimized versions and the frontend reports token savings.

D. *Validation and Inference Layer*

The semantic validator decides whether the optimized prompt is acceptable. I find this step important. If validation succeeds the optimized prompt can be sent to the target LLM. If validation fails the original prompt is used.

E. *Analytics Layer*

The analytics module can record token counts, reduction percentages, optimization latency, validation outcomes, and fallback events. These measurements are useful for evaluating whether optimization provides a net benefit.

V. Algorithm

Algorithm 1: TokenTrim Prompt Optimization
Input: Prompt P; Output: optimized prompt P' and token statistics

BEGIN
Read the user prompt P.
Count original tokens T_o.
Preprocess and analyze P.
Detect repeated low-value content.
Preserve task requirements, constraints, entities, facts and output format.
Generate optimized prompt P'.
Count optimized tokens T_c.
Compute token savings and reduction percentage.
Calculate semantic similarity S(P,P').
If S(P P') is greater than or equal to τ return P' else return P.
Report token statistics and validation status.
END

A. Complexity and Practical Considerations

In my view the running cost of optimization depends on the tokenizer, the redundancy detector, the embedding model and the optimization strategy. TokenTrim is useful when the saved LLM running cost and the usage cost justify the extra work of the optimization stage. For prompts, with little repetition skipping optimization may be more efficient.

VI. EXPERIMENTAL ANALYSIS

A. Evaluation Dataset

The recommended evaluation set contains prompts, from general question answering, programming, education, technical explanation, research, content generation and long-context tasks. Prompts should contain both concise examples and deliberately verbose examples so that the evaluation set can be tested for false compression and useful compression.

B. Performance Metrics

Metric	Purpose
Original token count	Baseline input size
Optimized token count	Input size after optimization
Token reduction (%)	Primary efficiency measure
Semantic similarity	Meaning preservation check
Optimization latency	Cost of the optimizer itself
Response quality	Downstream LLM quality
Fallback rate	Reliability of safe recovery

C. Experimental Procedure

- Run the original prompt through the target LLM and record output quality and timing.
- Run TokenTrim on the same prompt.
- Record original and optimized token counts.
- Measure semantic similarity and optimization latency.
- Send the optimized prompt to the same target LLM.
- Compare response quality with the original-prompt result.
- Repeat over multiple prompt categories.
- Calculate aggregate token savings and fallback rate.

D. Project Demonstration Outcome

TokenTrim is meant to cut away words from a prompt but still keep the user's request clear. I want to show how TokenTrim can keep the core meaning while removing fluff. The system also knows how to deal with pipeline failures. When a step, in the pipeline stops the system does not just fail quietly; it reports the issue. Uses a safe fallback. In the optimization workflow we first look for problems that can be fixed before the model runs. For example bad records are cleaned up early. Then we handle the tokenizer, the embedding, the loading of the model the API timeout and any machine-learning service failure. Each of these is. A safe fallback is used instead of producing a broken prompt. This way TokenTrim stays reliable..

E. Result Reporting

I think the system must produce numerical results. The report must not count example token reductions or carbon savings as results. The final report should contain the token reduction, the middle token reduction, the spread of semantic similarity the time spent on optimization the rate of fallback and a comparison of the quality of downstream responses.

VII. DISCUSSION

A. Advantages

- Reduces input tokens when there is redundancy.
- Shows before-and-after token counts for transparency.
- Operates at the application layer without changing the target LLM.
- Includes validation and safe fallback options.
- Can lower token-based API costs when input tokens are charged.
- May lead to responses and less compute use if the optimization effort is smaller, than the inference work saved.
- Offers a way to support Green AI by optimizing prompts.

B. Limitations and Scalability

TokenTrim does not promise a fixed percentage of cut because the prompt structure changes. I notice that some prompts are already short. Even if the meaning stays the same the results may not be alike. I also see that the optimizer uses computing power. The real advantage has to be measured instead of taken for granted. I know that carbon reduction depends on hardware, model design, data-center efficiency, batch size, usage level and the electricity mix.

C. Sustainability Impact

The sustainability argument relies on a chain of causes of a simple switch from tokens to CO2. If there are extra input tokens the system can process input faster. Faster processing can use energy. Using energy can lower emissions but only if the electricity comes from a clean source. Because of this TokenTrim should first show how tokens it saved. TokenTrim should only try to calculate energy use or carbon impact when it has the data, about the infrastructure.

VIII. CONCLUSION

TokenTrim offers a token optimisation layer for big language models. TokenTrim captures a prompt analyses use finds redundancy and creates a compact version while checking that meaning stays the same and it falls back, to the original prompt if needed. TokenTrim builds on established compression research and emphasises practical user interaction, transparency, reliability and independence from specific models.

The project also connects efficiency with sustainable AI and prompt efficiency is a key idea that links these two. Prompt efficiency is tied closely to sustainable AI and the project highlights this link. Reducing input is a concrete application-level optimization that can lower cost, lower latency, lower energy use and reduce emissions.. The claim of environmental benefit must be backed by measurements that also count the cost of the optimizer itself and prompt

efficiency must be measured carefully. TokenTrim offers both an engineering solution and an evaluation framework, for studying efficiency in large language models and TokenTrim is designed to help researchers see how prompt efficiency can be improved.

IX. PERFORMANCE COMPARISON

TokenTrim is meant to work along with not take the place of, research, about shortening prompts. The main thing that makes TokenTrim different is putting compression into a process that users can see and use. It includes ways to check if things are working and options to go back if needed.

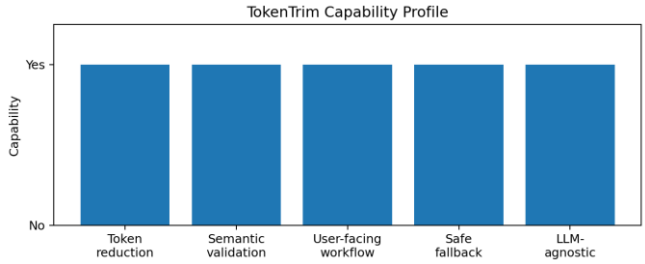


Fig. 2. TokenTrim capability profile.

Feature	Traditional LLM Use	TokenTrim
Token visibility	Usually limited	Explicit original/optimized counts
Prompt compression	Manual or absent	Automated
Semantic check	Not standard	Integrated validation
Failure fallback	Provider dependent	Original-prompt fallback
Cross-model layer	Provider specific	Application-layer design
Sustainability metric	Usually absent	Token/energy/carbon evaluation framework

I think the comparison is both architectural and qualitative. For the comparison to show superiority the comparison must be proven through controlled experiments that use the same prompts, target model, hardware and evaluation criteria.

X. FUTURE WORK

• Adaptive optimization modes for compression, balanced compression and aggressive compression.

• Model-aware optimization that works with tokenizers and different families of large language models.

• Local optimization models that help improve privacy.

• Real-time energy estimation and real-time carbon estimation using measured infrastructure telemetry.

• Integration with retrieval-augmented generation and with long-context applications.

• Learning-based optimization that uses user feedback.

• Automatic benchmark. Automatic regression testing.

• Provider-specific cost. Provider-specific optimization recommendations.

XII. REFERENCES

[1] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang and L. Qiu (2023). LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. Proc. EMNLP, 2023 pp. 13358–13376. DOI: 10.18653/v1/2023.emnlp-main.825. LLMLingua helps speed up language models by compressing prompts.

[2] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang and L. Qiu (2024). LongLLMLingua:. Enhancing LLMs in Long Context Scenarios via Prompt Compression. Proc. ACL, 2024 pp. 1658–1677. DOI: 10.18653/v1/2024.acl-long.91. LongLLMLingua expands on LLMLingua. Works well with long contexts.

[3] Z. Pan et al. (2024). LLMLingua-2: Data Distillation for Faithful Task-Agnostic Prompt Compression. Findings of ACL 2024 pp. 963–981. DOI: 10.18653/v1/2024.findings-acl.57. LLMLingua-2 builds on LLMLingua to keep prompts faithful across tasks.

[4] R. Schwartz, J. Dodge, N. A. Smith and O. Etzioni (2020). Green AI. Communications of the ACM, vol. 63, No. 12 Pp. 54–63. Green AI discusses the impact of AI systems.

[5] E. Strubell, A. Ganesh and A. McCallum (2019). Energy and Policy Considerations for Deep Learning in NLP. Proc. ACL, 2019 pp. 3645–3650. DOI: 10.18653/v1/P19-1355. This paper highlights how energy use affects NLP research.

[6] A. S. Luccioni, S. Viguier and A.-L. Ligozat (2023). Estimating the Carbon Footprint of BLOOM a 176B Parameter Language Model. Journal of Machine Learning Research, vol. 24 No. 253 Pp. 1–15. This study estimates the carbon cost of language models, like BLOOM.

[7] A. S. Luccioni, Y. Jernite and E. Strubell (2024). Power Hungry Processing: Watts Driving the Cost of AI Deployment? Proc. ACM FAccT, 2024 pp. 85–99. DOI: 10.1145/3630106.3658542. This paper explores how power consumption drives AI deployment costs.